Pravadh DocSign — Sample API Callouts

Package: Pravadh DocSign (namespace `prvd`), version 0.11.0 (`04tdN000000SNo5QAG`) · **Date:** 29 September 2026

1. No external API callouts

Pravadh DocSign makes **no HTTP callouts** to any external system. The package contains:

- no `HttpRequest` / `Http.send`, `WebServiceCallout` or `callout:` usage in Apex;
- no Remote Site Settings, Named Credentials, External Credentials, Connected Apps or External Services;
- no external JavaScript, CSS, CDN, analytics or telemetry. PDF.js 4.10.38 is bundled in the `DocSignPdfEngine` static resource.

Data never leaves the subscriber org except through standard Salesforce email (signing requests and notifications, sent with `Messaging.sendEmail`) and the external signer's own HTTPS session with the customer's Salesforce Site.

There are therefore no third-party request/response pairs to document. For completeness, the two browser requests the package makes to **the same Salesforce org** are documented below with all header values. Session and token values are redacted.

2. Same-org request 1: PDF download in the preparation editor

Where: `DocSignPdfPrep` Visualforce page (authenticated internal users only, embedded in the `docSignPdfPrep` LWC), file `docsign-prep-ui.js` in the `DocSignPdfEngine` static resource. **Why:** PDF.js needs the uploaded PDF's bytes in the browser. The `/sfc/servlet.shepherd` file URL redirects to the `*.file.force.com` content domain and is blocked by CORS, so the editor reads the file from the org's own REST API on the same Visualforce origin. **Controls:** runs only for a `068` ContentVersion id of the document being prepared; the request honours the user's own file access; the session ID is used only in this header and is never put in a URL, stored, logged or sent to another host.

Request

```
GET /services/data/v67.0/sobjects/ContentVersion/068XXXXXXXXXXXXXX/VersionData HTTP/1.1
Host: <mydomain>--prvd.vf.force.com
Authorization: Bearer 00DXXXXXXXXXXX!<session id redacted>
Accept: */*
Cache-Control: no-store
(credentials: 'omit' — no cookies are sent)
```

Response (success)

```
HTTP/1.1 200 OK
Content-Type: application/octetstream
Content-Length: 482113
Cache-Control: no-cache,must-revalidate,max-age=0,no-store,private

%PDF-1.7 ... (binary PDF bytes)
```

Response (no access / invalid session)

```
HTTP/1.1 404 Not Found            (or 401 Unauthorized)
Content-Type: application/json;charset=UTF-8

[{"errorCode":"NOT_FOUND","message":"The requested resource does not exist"}]
```

If this request fails, the editor falls back to the `@AuraEnabled` method `getPdfData` (files up to 2 MB, user mode) or asks the user to pick the same file locally (checked by size).

3. Same-org request 2: page images on the public signing page

Where: `DocSignSign` Visualforce page on the customer's Salesforce Site (external signer, guest user). It fetches one page image at a time from the `DocSignPageImage` page on the same Site. **Controls:** every request re-validates the signing token, envelope status, expiry and routing order, and returns an image only if the requested `068` version id is registered on that envelope's document. Otherwise it returns an empty body.

Request

```
GET /<site-path>/prvd__DocSignPageImage?t=<signing token redacted>&v=068XXXXXXXXXXXXXX HTTP/
1.1
Host: <customer-site>.my.salesforce-sites.com
Accept: */*
Cache-Control: no-store
Referer: (none — the page sends <meta name="referrer" content="no-referrer">)
(credentials: 'same-origin')
```

Response (valid token and page)

```
HTTP/1.1 200 OK
Content-Type: text/plain;charset=UTF-8
Cache-Control: no-cache, must-revalidate, max-age=0, no-store, private
Expires: Thu, 01 Jan 1970 00:00:00 GMT

/9j/4AAQSkZJRgABAQ... (base64 JPEG; the page accepts it only if it starts with the JPEG signa
ture)
```

Response (invalid token, wrong document or ended envelope)

```
HTTP/1.1 200 OK
Content-Type: text/plain;charset=UTF-8

(empty body)
```

4. Outbound email (not an API callout)

Signing requests, reminders and completion notices are sent with `Messaging.sendEmail` through Salesforce's own mail servers (`setSaveAsActivity(false)`, no Task or EmailMessage records). Messages contain the signer's name, envelope title, the sender's message and the signing link; completion emails attach the signed PDF and Certificate of Completion.